Digital Twin Trader

Juba Amari – Sorbonne Université, Licence de Mathématiques
Halfdan Feldthus – Københavns Universitet, BSc. Mathematics and Economics

April 2026

Abstract

We provide a digital twin framework for optimal execution when trading is costly and the trader has access to a return forecasting signal. We analyse optimal execution under a more general and underexplored trading cost model that encompasses both market impact and fixed trading costs. Most of the existing literature considers these costs in isolation, and practical implementation frameworks are limited. We formulate the problem in a dynamic programming setting, derive the associated Hamilton–Jacobi–Bellman (HJB) equations, and characterize optimal strategies under different cost specifications. Since the full model does not admit a closed-form solution, we propose a deep learning–based approximation obtained by minimizing the residual of the HJB equation using automatic differentiation. The framework is implemented as a digital twin of the market, allowing for the execution and evaluation of strategies on both historical and real-time data. Results on simulated data show significant improvements over standard benchmark strategies, giving a 8.3 Sharpe ratio improvement compared with the baseline on a signal with 0.1 Signal to Noise Ratio (SNR). The digital twin framework is finally used to test execution strategies on market data in different conditions. We hope that this work.

1 Introduction

Investors consistently able to predict future returns are often nonetheless unprofitable once they put their strategies into action. Namely, poor execution and the disregard of trading costs often turn otherwise fruitful strategies sour. We seek to answer how execution of trading strategies can be optimized in the face of trading costs, and how the solution can be implemented through the use of a digital twin system to aid real-time trading. To achieve a more accurate and sophisticated model of trading costs, we analyse a model that includes both fixed transaction costs and the impact costs associated with trading. This allows us to model an otherwise more general version of the problem that is suited for investors of any size. Here the literature has primarily only considered the problem with one of the cost types and therefore only large or small investors, respectively.

Our approach expands on the literature in two ways. First and foremost, we consider a model with both impact costs and fixed transaction costs. In general the literature solves the problem of trading with these costs in isolation. The first paper of its kind that solves the problem of purely impact costs was Geranlou & Pedersen [10] in 2013. The case of only linear costs has also been solved by Raj et al. and Capital Fund Management in 2012 [8]. The case of both cost types has been considered before, but no closed-form solution exists and current numerical methods are limited [16]. We provide a novel numerical method for solving that problem based on recent developments within deep learning for stochastic control.
Secondly, practical implementation frameworks are limited in the academic space. We provide a digital twin implementation that allows for execution of optimal trading. To the authors' knowledge, this has only been applied in the case of deterministic trades to be executed [1]. In the general optimal execution setting with predictable returns and transaction costs, our approach fits in the optimal execution literature as described in Table 1.

In the first section we describe the problem of optimal trading with predictable returns under trading costs in continuous time. This stochastic control problem is then formulated as a PDE

Table 1: Optimal execution under different transaction cost specifications

Paper	Cost Model	Approach
Geranlou & Pedersen [10]	Impact	Analytical solution
Capital Fund Management [8]	Fixed	Analytical solution
Capital Fund Management [16]	Fixed and small impact	Asymptotic analysis of HJB
This paper	Fixed and impact	Numerical solution via physics-informed neural networks

using a derivation of the HJB-equation. We then offer solutions under different conditions and then finally our approximation of the full model solution using a physics-informed neural network (PINN). The following section describes how the solution to the problem can be implemented to facilitate actual trading by discretization of the solution and giving estimation procedures of the different model parameters. In the third section we give an overview of how this implementation can be physically achieved through the use of a digital twin system. Lastly, we provide a snapshot of how the model performs under different conditions.

2 Optimal Trading

2.1 Problem statement

Let $(X,\mathcal{A},P)$ be a probability space. We consider an asset with price S_t that follow the dynamics

$$dS_t = (r + \mu_t)S_t dt + dM_t \tag{1}$$

Where μ_t is the instantaneous return in excess of the risk-free rate r and M_t is a noise term martingale (eg. brownian motion or a jump diffusion). We pose that the trader has access to a signal p_t that linearly predicts returns i.e.

$$\mu_t = p_t + \varepsilon_t$$

for some zero-mean noise term ε_t. Furthermore we assume that this signal follows an Ornstein-Uhlenbeck process (with mean 0 for ease of notation)

$$dp_t = -\psi p_t dt + \beta dW_t \tag{2}$$

With W_t a Brownian motion, where M_t and W_t are independent. We let x_t denote the proportion of the portfolio invested in the asset at time t. The remainder of the portfolio remains in the risk-free asset. Thus we can view $x > 1$ as lending at the risk-free rate, and investing in the asset. This implicitly assumes unconstrained perfect capital markets. We let the instantaneous rate of trading u_t be given by

$$dx_t = u_t\, dt$$

Let $\mathcal{F} \coloneqq (\mathcal{F}_t)_{t\leq T}$ be the natural filtration of the Brownian motion W_t, giving us the filtred space $(X, \mathcal{A},\mathcal{F},P)$. We impose the condition $u_t \in \mathcal{F}_t$.

We decompose the costs of trading in two: The temporary impact cost of trading, and the spread-cost. The costs associated with trading at rate $u_t = u$ is given by

$$\text{Spread Cost} = \gamma|u|$$

and

$$\text{Impact Cost} = \frac{\kappa}{2}u^2$$

This allows us to arrive that the instantaneous reward function, which is comprised of the expected return to be earned px and the total cost $\frac{\gamma}{2}|u| + \frac{\kappa}{2}u^2$

$$f(x, p, u) = xp - \lambda x^2 - \frac{\kappa}{2}u^2 - \gamma|u| \tag{3}$$

Where $\lambda > 0$ is a risk-aversion parameter. Finally the objective is to maximize the expected cumulative gain over a horizon $T \in \bar{\mathbb{R}}_+$, which can be written as the value function V defined as follows:

$$V(t,p,x) = \sup_{u \in \mathcal{U}} \mathbb{E}\left[\int_t^T e^{-\rho(s-t)} f(x_s, p_s, u_s)\, ds \;\middle|\; p_t = p, x_t = x \right]$$

where ρ is the discount rate, or risk-free interest rate. And u here is our control variable, for the integral to be well defined, we'll then assume $u \in \{u_t \in \mathcal{F}_t : (\int_X \int_{[t,T]} u_s^2 \; dsdP) < +\infty\} = \{u_t : \mathbb{E}\left[\int_t^T u_s^2 ds\right] < +\infty\}$

Equivalent to, $u \in \mathcal{U} \coloneqq \mathcal{L}^2(X \times [0,T], \mathcal{P}, P \otimes \ell)$
with $\mathcal{P}$ the predictable σ-algebra and ℓ the Lebesgue measure.

2.2 Derivation of the Hamilton-Jacobi-Bellman Equation

To derive the HJB equation, we start by applying the dynamic programming principle of optimality [7, 17], which states that the optimal strategy from time t onwards must be optimal for any subproblem starting at a later time. This leads us to the following expression for the value function:

$$\begin{aligned} V(t,p,x) &= \sup_{u \in \mathcal{U}} \mathbb{E}\left[\int_t^{t+dt} e^{-\rho(s-t)} f(x_s, p_s, u_s)\, ds + e^{-\rho dt} V(t+dt, p+dp, x+dx) \right] \\ &= \sup_{u \in \mathcal{U}} \mathbb{E}\left[f(x,p,u)\, dt + e^{-\rho dt} V(t+dt, p+dp, x+dx) \right] \end{aligned} \tag{4}$$

where dp and dx are the changes in the state variables over the infinitesimal time interval dt.

Next, we perform a Taylor series expansion of $V(t,p,x)$ to express the change in the value function over dt in terms of the changes in the state variables.
For this, we assume V is of class $\mathcal{C}^{1,2,2}([0,T],\mathbb{R}^2)$:

$$dV = \frac{\partial V}{\partial t} dt + \frac{\partial V}{\partial p} dp + \frac{\partial V}{\partial x} dx + \frac{1}{2} \frac{\partial^2 V}{\partial p^2} (dp)^2 + \frac{1}{2} \frac{\partial^2 V}{\partial x^2} (dx)^2 + \frac{\partial^2 V}{\partial p \partial x} dp\, dx$$

Higher-order terms are ignored as they vanish in the limit $dt \to 0$.

Next, we substitute the dynamics of p given in equation (1) into the expression for dV. By Itô's isometry [15], we have

$$(dp)^2 = \psi^2 p^2 (dt)^2 + \beta^2 (dW)^2 + 2\psi p \beta\, (dW)(dt).$$

Since $\mathbb{E}[(dW)^2] = dt$ and the cross term $2\psi p\beta\, (dW)(dt)$ is of order $dt^{3/2}$, we can ignore both the cross term and the $(dt)^2$ contribution in the limit $dt \to 0$, giving:

$$(dp)^2 = \beta^2\, dt.$$

As for dx, we have $dx = u\, dt$, so $(dx)^2 = u^2 (dt)^2$, which also vanishes as $dt \to 0$.

Substituting these into the expression for dV, we get:

$$dV = \left(V_t + \tfrac{1}{2}\beta^2 V_{pp} + V_x u - \psi p V_p\right) dt + \beta V_p\, dW$$

Now, using $V(t+dt, p+dp, x+dx) = V(t,p,x) + dV$, we substitute into (4):

$$\begin{aligned} V(t,p,x) &= \sup_{u \in \mathcal{U}} \mathbb{E}\left[f\, dt + e^{-\rho dt} (V + dV) \right] \\ &= \sup_{u \in \mathcal{U}} \mathbb{E}\left[f\, dt + (1 - \rho\, dt)(V + dV) \right] \\ &= \sup_{u \in \mathcal{U}} \mathbb{E}\left[f\, dt + V - \rho V\, dt + dV - \rho\, dV\, dt \right] \end{aligned}$$

The term $\rho\, dV\, dt$ is of order dt^2 and vanishes as $dt \to 0$. Here we have used $e^{-\rho dt} \approx 1 - \rho\, dt$ for small dt. Subtracting V from both sides and dividing by dt:

$$0 = \sup_{u \in \mathcal{U}} \left\{ f - \rho V + \frac{\mathbb{E}[dV]}{dt} \right\}$$

Using the linearity of the expectation operator and $\mathbb{E}[dW] = 0$ [15], we simplify:

$$\frac{\mathbb{E}[dV]}{dt} = V_t - \psi p V_p + \tfrac{1}{2}\beta^2 V_{pp} + V_x u.$$

Taking the limit $dt \to 0$, we obtain the HJB equation for this problem:

$$\rho V = \sup_u \left\{ f + V_t - \psi p V_p + \tfrac{1}{2}\beta^2 V_{pp} + V_x u \right\} \tag{5}$$

To simplify the notation, we define the Hamiltonian $\mathcal{H}$ as follows:

$$\mathcal{H}(t, p, x, u, V_p, V_{pp}, V_x) = f(x, p, u) - \psi p V_p + \tfrac{1}{2}\beta^2 V_{pp} + V_x u$$

so the HJB equation can be rewritten as:

$$\rho V = V_t + \sup_u \{\mathcal{H}(t, p, x, u, V_p, V_{pp}, V_x)\} \tag{6}$$

2.3 Boundary Conditions

To solve the HJB equation, we need to specify the boundary conditions for a finite horizon T. For the rest of this paper, we assume V a viscosity solution of (6),it must satisfy the terminal condition:

$$V(T, p, x) = 0$$

This is because at time T, there are no further rewards, so the value function is zero. The state variables p and x are both unbounded, as p can take any real value and x can be long or short. In summary:

$$V(T, p, x) = 0 \quad \text{for all } p, x \tag{7}$$

$$p \in (-\infty, +\infty) \tag{8}$$

$$x \in (-\infty, +\infty) \tag{9}$$

2.4 Optimal Control

2.4.1 Case $\kappa = 0$, $\gamma = 0$

In the case where there are no transaction costs, the optimal control can be found by taking the first-order condition of the HJB equation with respect to u. The problem simplifies to the pointwise optimization:

$$\sup_u \left\{ xp - \lambda x^2 + u V_x \right\} \tag{10}$$

Since $\mathcal{H}$ is linear in u, the optimal control is:

$$u^* = \begin{cases} +\infty & \text{if } V_x > 0 \\ -\infty & \text{if } V_x < 0 \\ \text{any value} & \text{if } V_x = 0 \end{cases} \tag{11}$$

This is degenerate in practice: without transaction costs there is no bound on position size. This motivates the introduction of transaction costs to ensure a well-defined and finite optimal control.

2.4.2 Case $\kappa > 0$, $\gamma = 0$

In this section, we consider the case where the transaction cost is purely quadratic [10], corresponding to $\gamma = 0$. This implies that the control problem is smooth, and we can expect the optimal trading strategy to be a continuous function of the state variables [18]. Starting from the HJB equation derived in the previous section, the value function $V(t,p,x)$ satisfies

$$\rho V = \sup_u \left\{ xp - \lambda x^2 - \frac{\kappa}{2}u^2 + V_x u - \psi p V_p + \frac{1}{2}\beta^2 V_{pp} \right\}, \tag{12}$$

where ρ is the discount rate, V_x and V_p denote the partial derivatives of V with respect to x and p, respectively, and the last two terms account for the drift and diffusion of the predictor p.

Only two terms in the HJB involve u, namely $-\frac{\kappa}{2}u^2 + V_x u$. Since this is a concave quadratic function of u, the first-order condition is necessary and sufficient for optimality. Differentiating with respect to u yields

$$0 = -\kappa u + V_x,$$

which immediately gives the optimal control:

$$u^* = \frac{V_x}{\kappa}. \tag{13}$$

This result has a clear economic interpretation: the agent increases her position proportionally to the marginal value of holding the asset, while the quadratic transaction cost κ penalizes large trades [18].

Substituting u^* back into the HJB eliminates the control variable:

$$-\frac{\kappa}{2}(u^*)^2 + V_x u^* = \frac{V_x^2}{2\kappa}.$$

Therefore, the HJB equation reduces to

$$\rho V = xp - \lambda x^2 + \frac{V_x^2}{2\kappa} - \psi p V_p + \frac{1}{2}\beta^2 V_{pp}.$$

The structure of the problem suggests a quadratic ansatz for the value function [17, 18]. Since the running reward is quadratic in x, the dynamics are linear, and the control enters linearly in the HJB, we write:

$$V(p,x) = Ax^2 + Cxp + Dp^2 + F, \tag{14}$$

with constants A, C, D, F to be determined. Computing the derivatives:

$$V_x = 2Ax + Cp, \qquad V_p = Cx + 2Dp, \qquad V_{pp} = 2D. \tag{15}$$

The term $\frac{V_x^2}{2\kappa}$ expands to

$$\frac{V_x^2}{2\kappa} = \frac{(2Ax + Cp)^2}{2\kappa} = \frac{2A^2}{\kappa}x^2 + \frac{2AC}{\kappa}xp + \frac{C^2}{2\kappa}p^2.$$

Similarly, $-\psi p V_p = -\psi C xp - 2\psi D p^2$ and $\frac{1}{2}\beta^2 V_{pp} = \beta^2 D$.

Matching coefficients in powers of x and p yields the following system of algebraic equations [17]:

$$\rho A = -\lambda + \frac{2A^2}{\kappa}, \tag{16}$$

which is a Riccati equation in A. The economically relevant solution corresponds to the negative root, ensuring concavity of V in x.

$$\rho C = 1 + \frac{2AC}{\kappa} - \psi C, \qquad \text{or equivalently} \qquad C = \frac{1}{\rho + \psi - 2A/\kappa}.$$

$$\rho D = \frac{C^2}{2\kappa} - 2\psi D, \qquad \text{so that} \qquad D = \frac{C^2}{2\kappa(\rho + 2\psi)}.$$

$$\rho F = \beta^2 D, \qquad \text{giving} \qquad F = \frac{\beta^2 D}{\rho}.$$

The optimal control becomes

$$u^*(p,x) = \frac{V_x}{\kappa} = \frac{2A}{\kappa}x + \frac{C}{\kappa}p. \tag{17}$$

This policy is linear in both the position and the predictor [17, 18, 15]. The term proportional to x induces mean-reversion in the position, limiting inventory risk, while the term proportional to p trades in the direction of the predictive signal, exploiting expected returns.

2.4.3 Case $\kappa > 0$, $\gamma > 0$

This section is under the assumption that $\frac{\partial \mathcal{H}}{\partial u} = 0$, which gives an explicit formula for u^*.

To find the optimal control u^*, we take the first-order condition of the HJB equation with respect to u, as the problem simplifies to the pointwise optimization:

$$\sup_u \{f + uV_x\} \tag{18}$$

since these are the only terms in the HJB equation that depend on u.

f is differentiable with respect to u everywhere except at $u = 0$, where the term $\gamma \mid u \mid$ is not differentiable. We therefore consider two cases:

$$\frac{\partial f}{\partial u} = -\kappa u - \gamma \quad \text{for } u > 0$$
$$\frac{\partial f}{\partial u} = -\kappa u + \gamma \quad \text{for } u < 0$$

We also have $\frac{\partial}{\partial u}(uV_x) = V_x$, so the first-order conditions are:

$$0 = -\kappa u - \gamma + V_x \quad \text{for } u > 0$$
$$0 = -\kappa u + \gamma + V_x \quad \text{for } u < 0$$

This gives:

$$u^* = \frac{V_x + \gamma}{\kappa} \quad \text{for } u > 0 \tag{19}$$
$$u^* = \frac{V_x - \gamma}{\kappa} \quad \text{for } u < 0 \tag{20}$$

Consistency of each case requires $V_x > \gamma$ when $u > 0$ and $V_x < -\gamma$ when $u < 0$. When $|V_x| \leq \gamma$, neither case is consistent, and the optimal control is $u^* = 0$. This is the no-trade region, a standard feature of models with linear transaction costs [18].

We finally have:

$$u^* = \begin{cases} \dfrac{V_x + \gamma}{\kappa} & \text{if } V_x > \gamma \\ 0 & \text{if } -\gamma \leq V_x \leq \gamma \\ \dfrac{V_x - \gamma}{\kappa} & \text{if } V_x < -\gamma \end{cases} \tag{21}$$

The HJB equation can now be written as:

$$\rho V = V_t - \psi p V_p + \frac{1}{2}\beta^2 V_{pp} + \max_u \{f + V_x u\} \tag{22}$$

2.5 Solution

2.5.1 Case $T = \infty$

In the infinite horizon case, we look for a stationary viscosity, meaning that V does not depend on time:

$$\rho V = -\psi p V_p + \frac{1}{2}\beta^2 V_{pp} + \max_u \{f + V_x u\} \tag{23}$$

We define a residual neural network V_θ with an input projection layer of m units followed by n residual hidden layers of m units, and a scalar output layer. The parameter space is $\theta \in (\mathcal{M}_{m,m}(\mathbb{R}))^n$. The input $(p, \pi) \in \mathbb{R}^2$ is projected to $\mathbb{R}^m$ by the input layer, so no zero-padding is required.

$$\begin{aligned}
V_{\theta,1} &= \phi_1\left(x \cdot W^{(0)}\right) \\
&\vdots \\
V_{\theta,i} &= V_{\theta,i-1} + \phi_i\left(V_{\theta,i-1} \cdot W^{(i)}\right), \quad i = 2, \ldots, n \\
&\vdots \\
V_\theta &\coloneqq V_{\theta,n} \cdot W^{(\text{out})}
\end{aligned}$$

We use residual connections $V_{\theta,i} = V_{\theta,i-1} + \phi(V_{\theta,i-1} \cdot W^{(i)})$ rather than a standard MLP. This architecture, introduced by He et al.[11], mitigates vanishing gradients during backpropagation; which is particularly important here since training requires computing second-order derivatives V_{pp} of the network output.

with all $\phi_i : \mathbb{R}^m \to \mathbb{R}^m$ continuous smooth activation functions.

Our objective is then to approach a parameter space θ such that:

$$\begin{aligned}
&Res(V_\theta) = \rho V_\theta - (f - \psi p (V_\theta)_p + \frac{1}{2}\beta^2 (V_\theta)_{pp} + (V_\theta)_x u) \\
&\theta \coloneqq argmin_\theta\{Res(V_\theta)\}
\end{aligned}$$

We note that θ itself might not exist as V_θ has no reason to be convex or admit a global minimum with respect to θ.

Let p,$x \in \mathbb{R}^N$ for $N \in \mathbb{N}$, let r be a divisor of N we define the r steps training algorithm as follows:

- initialize $\theta = \theta_0$ a random parameter space
- for $k \leq r$ we take $(p_i,x_i)_{\frac{k}{N} \leq i \leq \frac{k+1}{N}}$ as the current batch, and for all $(x_i,p_i)_i$:
  - we compute $V = V_{\theta_i}(p_i,x_i)$ and $\tilde{V} = V_{\theta_i}(-p_i, -x_i)$
  - we compute $HJBL = Res(V)$
  - we compute $SL = (u^*(V_x) - u^*(\tilde{V}_x))^2$
  - we compute $SML = (V_p)^2 + (V_{px})^2$
  - we have $\mathcal{L}_{\theta_i} = HJBL + SL + SML$ and we update θ_i with AdamW

Given a loss $\mathcal{L}(\theta)$, AdamW [13] updates the parameters at each step k as follows:

$$\begin{aligned}
g_k &= \nabla_\theta \mathcal{L}(\theta_k) \\
m_k &= \beta_1 m_{k-1} + (1 - \beta_1) g_k \\
v_k &= \beta_2 v_{k-1} + (1 - \beta_2) g_k^2 \\
\hat{m}_k &= \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k} \\
\theta_{k+1} &= \theta_k - \eta_k \left(\frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon} + \lambda \theta_k\right)
\end{aligned}$$

where η_k is the learning rate, β_1, β_2 are moment decay rates, ϵ is a numerical stabilizer, and λ is the weight decay coefficient. The key distinction from Adam is the $\lambda\theta_k$ term, which applies weight decay directly to the parameters rather than incorporating it into the gradient, leading to better regularization.

η_k follows a cosine decay with warm restarts [12]:

$$\eta_k = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{T_{cur}}{T_i}\pi\right)\right)$$

$\eta_{\max}$ is the initial learning rate, $\eta_{\min}$ is the minimum learning rate, T_i is the number of steps in the i-th cycle, and T_{cur} is the number of steps since the last restart. After each restart, the cycle length is multiplied by t_{mul} and the peak learning rate by m_{mul}, allowing the optimizer to escape local minima while progressively refining the solution.

empirical simulations Now that we have an algorithm that should approach numerically the value function V, we can implement the algorithm in python to test empirically its convergence and the optimality of the function fitted.
We used Keras, the high-level API for Tensorflow to create a model training via the algorithm presented earlier (all code is available in the project repository), the parameters used are the following:
for the OU process: $\psi = 0.1$ and $\beta = 0.1$
for the economical setting: $\gamma = 10^{-3}$, $\kappa = 10^{-2}$, $\rho = 5\%$, $\lambda = 1$
We also set $m = 64$ and $n = 5$ and $\forall i \leq n \quad \phi_i = tanh$.

After 30 epochs of training at 2000 steps per epoch, we get a network V_θ that can approximate V, using automatic differentiation [6] to compute all derivatives through tensorflow calls of the GradientTape function [3]; we can still, once the model is trained, differentiate at every state (p,x) with respect to x and use $(V_\theta)_x$ to compute the optimal policy u^*.

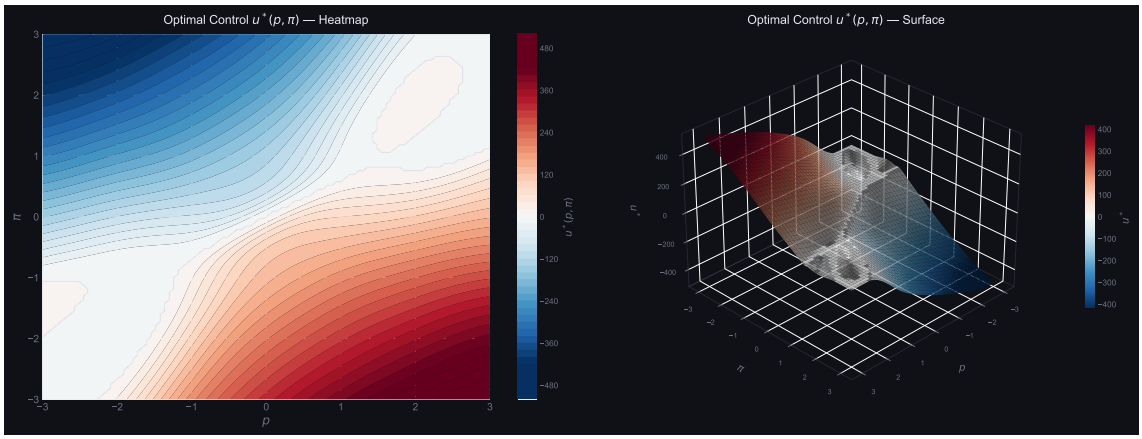

Figure 1: u^* given by values of (p, π)

We backtest our model on a synthetically generated data, we generate a synthetic OU process p_t of parameters ψ,β and we definer returns at time t: $r_t = p_t + \text{noise}$.The results of the model trading on 10 000 steps of the signal is given in

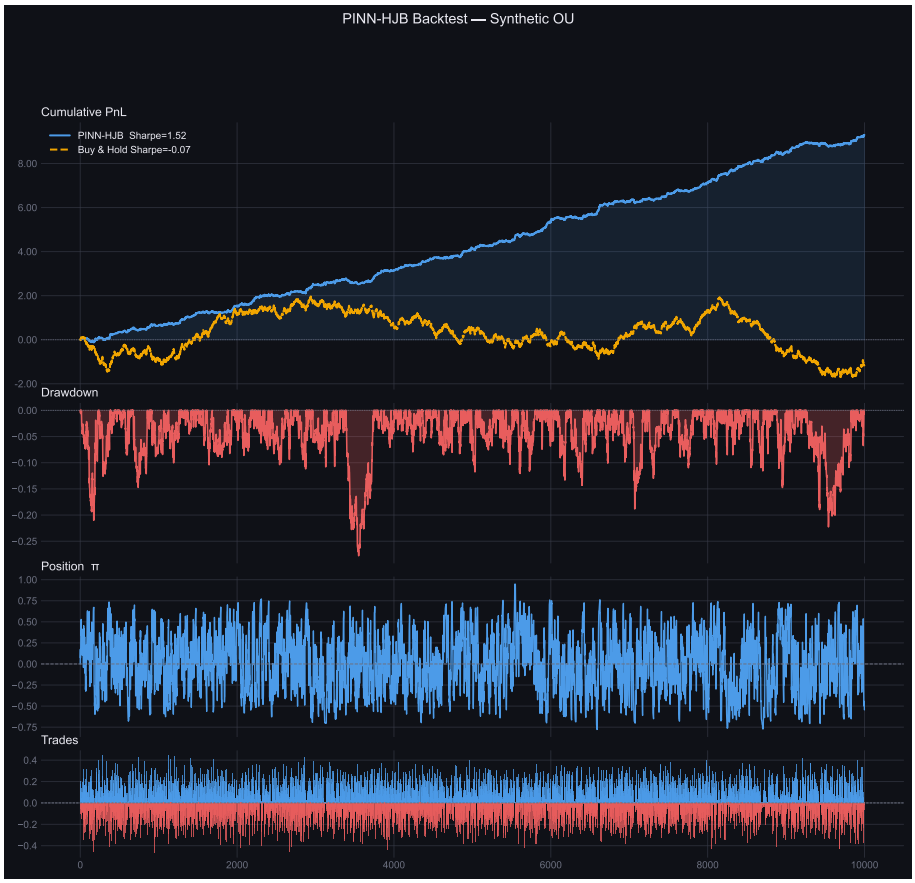

Figure 2: backtest on $10k$ signal steps

Finally, We design two strategies to compare our model with:

- **The proportional strategy:** at every step we trade the signal strength, i.e at every time t $u_t = p_t$
- **The greedy strategy:** at every step we trade $u_t = p_t/\kappa$ (greedy meaning we don't take into account the risk of having an inventory far from 0)

Across 200 simulated Monte-Carlo paths, we let these two strategies and our model trade for 2000 steps on each path, and we get the following results:

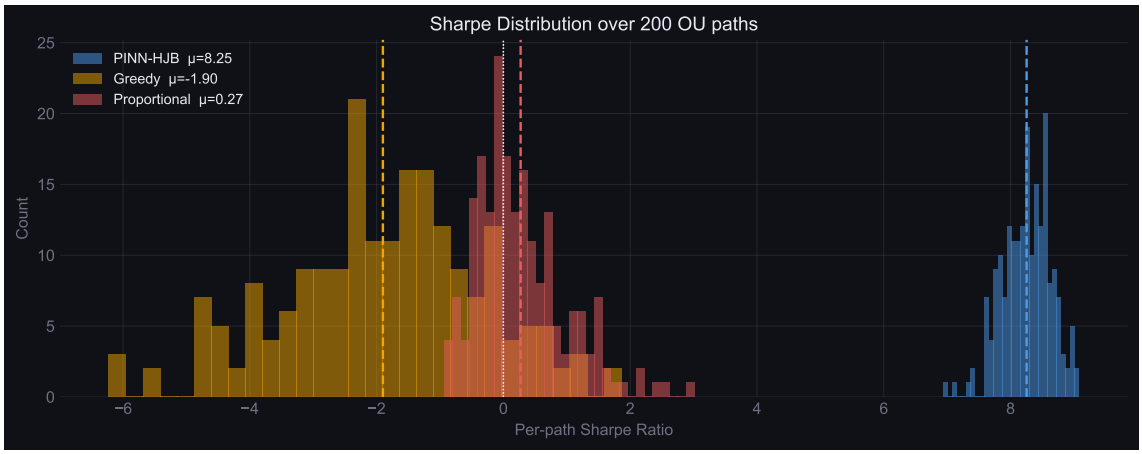

Figure 3: 3 strategies on 200 MC paths

Strategy	Mean Sharpe	Std	Min	Max	% > 0
PINN-HJB	8.253	0.378	6.944	9.079	100.0%
Greedy	-1.901	1.575	-6.223	1.862	10.5%
Proportional	0.271	0.708	-0.931	3.010	57.5%

Table 2: Annualised Sharpe ratio statistics over 200 synthetic OU paths

We can see than our model clearly outperforms both strategies by far, while both strategies fail to exploit the signal and end up with no edge, the model consistently has a positive Sharpe ratio with an average of ≈ 8.3.

Author note : A signal with such a high Sharpe ratio would be unrealistic in real life, or fast-decaying because of market efficiency, where actors in the market would rapidly find it, use it and exploit it until the market adapts.

2.5.2 Case $T < \infty$

The finite horizon case is more complex due to the time dependence of the value function. We use the deep learning method proposed by Dupré and Hainaut (2025) [9], which approximates both the value function V and the optimal control u^* with neural networks trained using the HJB equation as a loss.

We define our two neural networks as follows:

$$\begin{aligned} V_\theta &\approx V \\ u_\omega &\approx u^* \end{aligned} \tag{24}$$

We define the residual Hamiltonian

$$\tilde{\mathcal{H}}(\theta,\omega) = (V_\theta)_t + f(x,p,u_\omega) + (V_\theta)_x \cdot u_\omega - \psi p (V_\theta)_p + \frac{1}{2}\beta^2 (V_\theta)_{pp} - \rho V_\theta \tag{25}$$

At the true solution, i.e $V_\theta = V$ we have $\tilde{\mathcal{H}} = 0$ everywhere this is the quantity both networks are trained to zero out.

We define the following losses:

$$\mathcal{L}_1(\theta,\omega^{(k)}) = \mathbb{E}\left[\tilde{\mathcal{H}}(\cdot \mid \theta,\omega^{(k)})^2\right] + \mathbb{E}\left[(V_\theta(T,p,x)^2)\right] \tag{26}$$

$$\mathcal{L}_2(\theta^{(k+1)},\omega) = \mathbb{E}\left[\tilde{\mathcal{H}}(\cdot \mid \theta^{(k+1)},\omega)\right] \tag{27}$$

The algorithm presented consists of initializing initial weights, $\theta^{(0)}, \omega^{(0)} \in \mathbb{R}^n$, and an $\epsilon > 0$, At each step, two operations are performed:

- We first optimize the $V_{\theta^{(k)}}$ parameters by having

$$\theta^{(k+1)} = argmin_\theta \mathcal{L}_1(\theta,\omega^{(k)}) \tag{28}$$

- We can then set $V_{\theta^{(k)}} \leftarrow V_{\theta^{(k+1)}}$ and use this to optimize $u_{\omega^{(k)}}$ by using:

$$\omega^{(k+1)} = argmin_\omega \mathcal{L}_2(\theta^{(k+1)},\omega) \tag{29}$$

  And we set $u_{\omega^{(k)}} \leftarrow u_{\omega^{(k+1)}}$.

We repeat these steps until we have $\mid V_{\theta^{(k)}} - V_{\theta^{(k-1)}} \mid < \epsilon$.

Author note: We didn't get any similar results to 2.5.1 as of the time this paper is written, but we consider this an interesting idea to explore. As we tried recreating the algorithm with Keras, we have run into some problems during training that made results too slow for them to be in this paper before its deadline.

3 Model Execution: From Model to Trading

The investor might only have access to a raw trading signal with return-predicting features. The question then remains how to estimate and calibrate the remaining parameters from the problem statement before we can implement the solution.

3.1 Costs Estimation

The parameter γ represents the fixed cost of trading. Assuming that the trader pays no other broker-related fees, this can simply be set to half the bid-ask spread:

$$\gamma = \frac{\text{Spread}}{2S}.$$

Otherwise, γ can be increased to incorporate additional trading fees. Market impact costs are more involved. Empirically, they are often best described by a square-root law, i.e. the relative price change from trading Q shares scales as $\sqrt{Q}$, which would yield a cost term of degree $3/2$ in u rather than the quadratic term in (3). To remain within the quadratic cost model, we follow Almgren–Chriss [5], who use a similar model to ours. They further provide a useful heuristic to estimate κ: trading one percent of the total daily dollar volume incurs a price impact approximately equal to the bid-ask spread.

If we let M be the total wealth under management and V the daily market dollar volume traded per unit time, the participation rate is therefore $M|u|/V$, and the heuristic then gives the relative price impact $\Delta S/S$ as

$$\frac{\Delta S}{S} \approx 100 \frac{\text{Spread}}{S} \frac{M|u|}{V}.$$

The impact cost relative to wealth is this price impact multiplied by the fraction of wealth traded over the unit time, $|u|$, giving a cost rate of

$$\text{Impact Cost} = \frac{\Delta S}{S}|u| \approx 100 \frac{\text{Spread}}{S} \frac{M}{V} u^2.$$

This now coincides with the quadratic trading cost in (3), with

$$\kappa = 100 \frac{\text{Spread}}{S} \frac{M}{V}.$$

3.2 Discretely Trading a Continuous Model

Although our problem statement is formulated in continuous time, the investor (and corresponding digital twin) can only observe and act at discrete time points. As such, we separate the estimation in discrete time from the control problem, which is solved in continuous time. We then apply the resulting solution as an approximation in the discrete setting.

This means we have to consider an investor acting on a fixed time grid $\{t_k = k\Delta t \mid k = 0, \ldots, N\} \subset [0,T]$, observing the return-predicting signal $(p_k)_{k=0}^N$, which is $\mathcal{F}_{t_k}$-measurable. To embed this as an extension of the continuous-time model, we assume that $(p_k)_{k=0}^N$ follows an AR(1) process

$$p_{k+1} = \phi p_k + \eta_{k+1},$$

with $(\eta_k)_{k=1}^N$ i.i.d. $N(0,\sigma_\eta^2)$ and $0 < \phi < 1$, to be estimated via OLS on historical observations of the signal.

To find the optimal control, we must embed these observations as realizations of the continuous-time process (2). Using the stated conditional distributions given in [4], we have the following identities that give the continuous-time counterparts of the parameters:

$$\psi = -\frac{\ln(\phi)}{\Delta t},$$

and

$$\beta = \sigma_\eta \sqrt{\frac{2\psi}{1-\phi^2}}.$$

We have throughout this text posited that the investor has access to a return-predicting signal. In reality, signals that predict future returns often do so with a high degree of noise and may be short-lived in practice [14]. Nevertheless, an investor may attempt to construct such a signal by using a candidate signal that they believe drives future returns. They can then regress the signal on future returns on a rolling window basis to assess the signal's viability and determine its scale.

3.3 Backtesting

We can now solve the optimal trading problem as described in Section 2, giving us $u^*(t_k,p_k,x_k)$, where we let the portfolio weight evolve with the piecewise constant trading rate $\tilde{u}_k = u^*(t_k,p_k,x_k)$ for $t \in [t_k, t_{k+1})$ as

$$x_{k+1} = x_k + u_k \Delta t.$$

We can now test the performance of the strategy over time as follows. Given historical returns $(r_1, \ldots, r_N)$ and accounting for trading costs, the realized profit and loss (PnL) over the step from k to $k+1$ is

$$\mathrm{PnL}_k = x_k \, r_{k+1} - \gamma |\tilde{u}_k| \Delta t - \frac{\kappa}{2} (\tilde{u}_k)^2 \Delta t.$$

Performance can then be evaluated using standard metrics such as the Sharpe ratio:

$$\mathrm{SR} = \frac{\mu_{\mathrm{PnL}}}{\sigma_{\mathrm{PnL}}} \sqrt{\frac{1}{\Delta t}},$$

where μ_{PnL} and σ_{PnL} are the sample mean and standard deviation of the PnL.

4 The Digital Twin Trader

4.1 Description of System

The system's purpose is to allow an investor to seamlessly deploy a digital trader that executes a given trading strategy optimally. The investor must first set up the trading environment, namely by choosing the asset to trade, the time frame, the level of risk aversion, the cost models to consider, and lastly the execution strategy and signal.

The system then collects historical asset prices and estimates all relevant model parameters via the methods explained in the previous section. Based on the trading environment and the estimated parameters, the system finds the optimal trading-rate function $u^*(x,p)$ as explained in the second section. It then runs a backtest on the available data as described previously.

This backtest shows how the optimally executed strategy would have performed historically. It presents different performance metrics and the evolution of the position over time, along with the cumulative return. Lastly, the system allows for live updates of the position and performance as the strategy is implemented on incoming data. This allows the investor to be assisted in their actual trading.

As a further consideration, through the use of electronic markets and developer-centric brokers, the system could easily be expanded to allow the digital twin to deploy real trades dynamically. To illustrate how the system might be used in practice, it comes preloaded with different signals for testing. One such signal is a momentum signal defined as an exponential moving average over past returns.

The overall software implementation is built with modularity in mind, as described below. This allows investors to easily implement or upload their own signals for testing optimal execution.

4.2 Data

The data layer of the app uses historical asset prices and other measures of a given stock over a specified time frame. This market data is dynamically fetched using the Alpaca API. Live data is gathered from the Alpaca WebSocket, which continuously pushes new price information as it becomes available.

Furthermore, the API allows us not only to pull equity data, but we have also incorporated cryptocurrencies as an asset class. A major advantage of the Alpaca API is that it provides

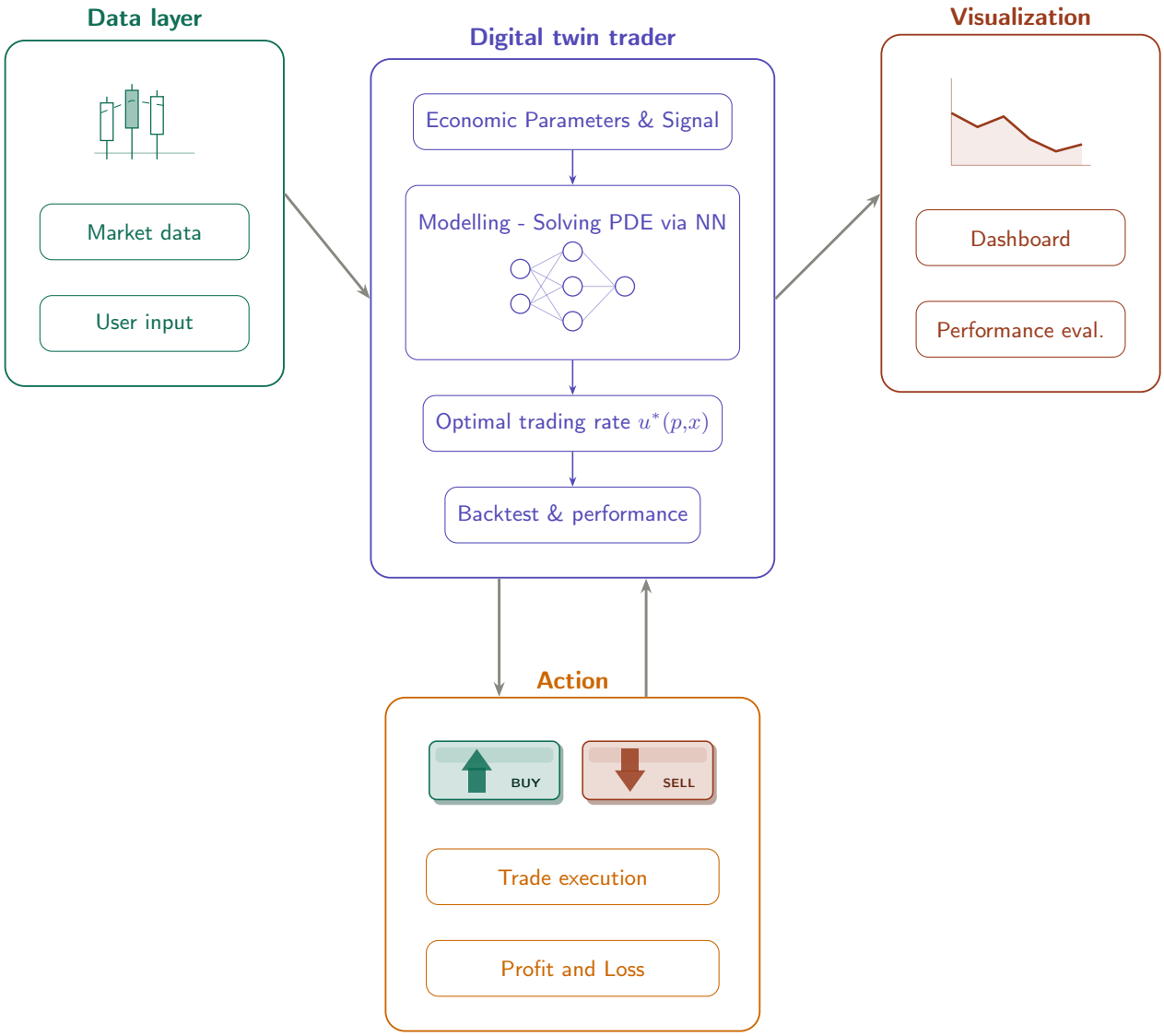

Figure 4: Digital Twin Trader System Overview

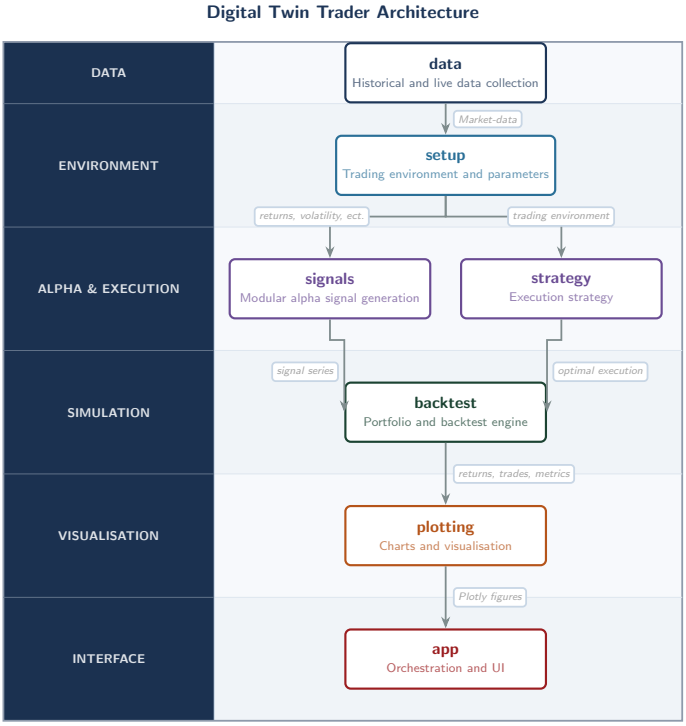

Figure 5: Overview of the implementation architecture

price information on the entire US equity market. This means that stocks with smaller market capitalizations, and in turn lower liquidity, can be considered. It is precisely when trading such assets that optimizing for costs becomes essential, as transaction costs are higher for low-liquidity assets.

timestamp	symbol	open	high	low	close	volume	trade_count	vwap
2026-01-02 19:00:00+00:00	CWCO	34.740000	34.840000	34.740000	34.840000	233.000000	3.000000	34.790000
2026-01-02 20:00:00+00:00	CWCO	34.840000	34.840000	34.725000	34.725000	401.000000	5.000000	34.776250
2026-01-05 14:00:00+00:00	CWCO	34.990000	34.990000	34.990000	34.990000	100.000000	1.000000	34.990000
2026-01-05 16:00:00+00:00	CWCO	34.875000	34.875000	34.875000	34.875000	100.000000	1.000000	34.875000
2026-01-05 17:00:00+00:00	CWCO	34.905000	34.905000	34.905000	34.905000	127.000000	6.000000	34.905000
2026-01-05 19:00:00+00:00	CWCO	34.900000	34.900000	34.900000	34.900000	103.000000	2.000000	34.900000

Table 3: Example of historical market-data of Consolidated Water INC. received from Alpaca's trading API

timestamp	open	high	low	close	volume	vwap
2026-04-13 12:43:00+00:00	70805.805000	70826.015000	70799.355000	70820.475000	0.000000	70812.685000

Table 4: Example of a live trading bar of Bitcoin from the Alpaca websocket

4.3 Software Implementation and Design

The app first collects the required inputs from the user, such as cost parameters, the signal, the asset, and so on. Market data is then fetched via the Alpaca API, which returns the asset's historical price movements. From there, the app computes the optimal execution strategy and subsequently visualizes a backtest over the historical period.

With the click of a button, the investor can then run a live trading test, which collects recent market data and runs the strategy on this warm-up period before displaying trades in real time.

The app's implementation follows a modular and object-oriented approach. The main visual layout and organization are built using the Streamlit framework [2], and Plotly is used for dynamic and interactive plots. The backend consists of modules for each of the trading-related components considered.

As shown in 5, the orchestrator first pulls market data and user inputs, which are then passed to the trading setup. Using the trading environment, the signal is produced, along with the initialization of the optimal execution algorithm. These components are then passed to a backtesting engine that computes historical or live performance and evaluates performance metrics, which are finally displayed in the app.

The visual layout is designed with functional simplicity in mind, ensuring that the user is presented with the most relevant information while remaining visually uncluttered.

4.4 Results and Performance Metrics: Backtests

We now turn to observing how the system performs under different conditions. To purely highlight the cost optimization of the intelligent digital twin, we use a signal comprised of future returns and samples from an OU process with variance 0.1 and mean reversion 0.1. As we consider trading at a 1-hour frequency, the signal's half-life becomes $\frac{\ln 2}{0.1} \approx 6.93$, or roughly one trading day.

We begin by observing how the digital twin performs when trading the strategy on a very liquid stock—in this case Alphabet Inc. (ticker GOOGL), a.k.a. Google. The digital twin collects the historical asset price of GOOGL over a period starting from 2025-07-01 to 2026-03-01. We now divide the backtest in terms of solutions to the optimal trading problem under either a regime with fully linear or quadratic costs, respectively.

We set each of the parameters to 1 bps, corresponding to $\gamma = 10^{-4}$ and $\kappa = 0$, or $\gamma = 0$ and $\kappa = 10^{-4}$ in the linear and quadratic cases, respectively. We set the risk aversion in accordance with an annual volatility target of 10% ex costs and the risk-free rate at a constant annual 2%. We then compare the performance of the execution strategies described in Sections 2.4.2 (referred to as Gârleanu–Pedersen, the authors who first formulated it) and 2.4.3 (referred to as the No-Trade Band), with the optimal portfolio without transaction costs (Markowitz), defined as $x_t = \frac{p_t}{\lambda}$ for the signal p_t.

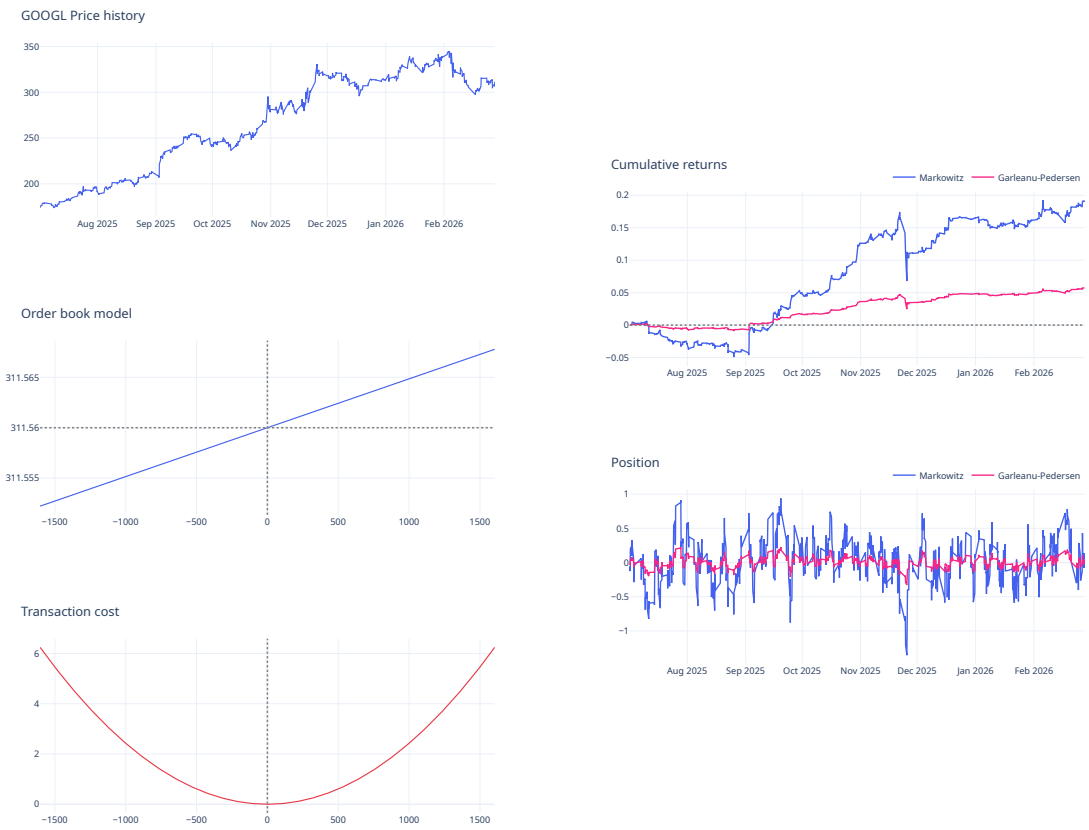

Figure 6: Comparison of the no-transaction-cost (Markowitz) portfolio with the optimal quadratic transaction cost (Gârleanu–Pedersen) portfolio in a purely impact cost setting. The left shows the implied order book model from the cost parameters with AUM of $10M, and the right shows the total cumulative return over the period along with position sizing over time.

	Ann. Excess Return	Ann. Vol	Sharpe	Max DD
Markowitz	0.0954	0.1039	0.9175	-0.0946
Garleanu-Pedersen	0.0238	0.0255	0.9344	-0.0237

Table 5: Net-of-cost performance comparison of the portfolio strategies: the no-cost (Markowitz) portfolio and the impact-cost-optimal (Gârleanu–Pedersen) portfolio, in terms of arithmetic annualized excess return, volatility, Sharpe ratio, and maximum drawdown.

Figure 7: Comparison of the no-transaction-cost Markowitz portfolio with the optimal No-Trade Band solution in a purely linear cost regime. The left shows the implied trading band and position dynamics, while the right shows cumulative returns over the period along with position sizing over time.

	Ann. Excess Return	Ann. Vol	Sharpe	Max DD
Markowitz	0.0803	0.1039	0.7725	-0.0947
No-Trade Band	0.0841	0.0947	0.8877	-0.0832

Table 6: Net-of-cost performance comparison under a linear transaction cost specification, contrasting the frictionless Markowitz portfolio with the optimal No-Trade Band policy. Reported statistics are arithmetic annualized excess return, volatility, Sharpe ratio, and maximum drawdown.

From Figure 6 and Table 5, we see that the optimal quadratic portfolio slightly outperforms in terms of Sharpe ratio, which is to be expected given that the very liquid stock carries low trading costs. The overall return is lower for the optimal quadratic strategy because the model prefers smaller position sizes relative to signal strength to avoid large quadratic trading costs. This is the main reason why it undershoots the volatility target of 10% by a small margin.

Similarly, looking at Figure 7 and Table 6, we see that the optimal linear portfolio slightly outperforms, while choosing a similar volatility level, as larger positions no longer incur additional costs.

Next, we turn to a situation where these cost-optimized portfolios really shine, namely in the presence of larger transaction costs associated with less liquid assets. We now consider the stock of Consolidated Water Co. (ticker CWCO). This smaller utilities company is far less liquid, with an average daily volume of around a few million dollars. We therefore keep the other parameters the same, but set $\gamma = 0.02$ and $\kappa = 0$, or $\gamma = 0$ and $\kappa = 0.05$.

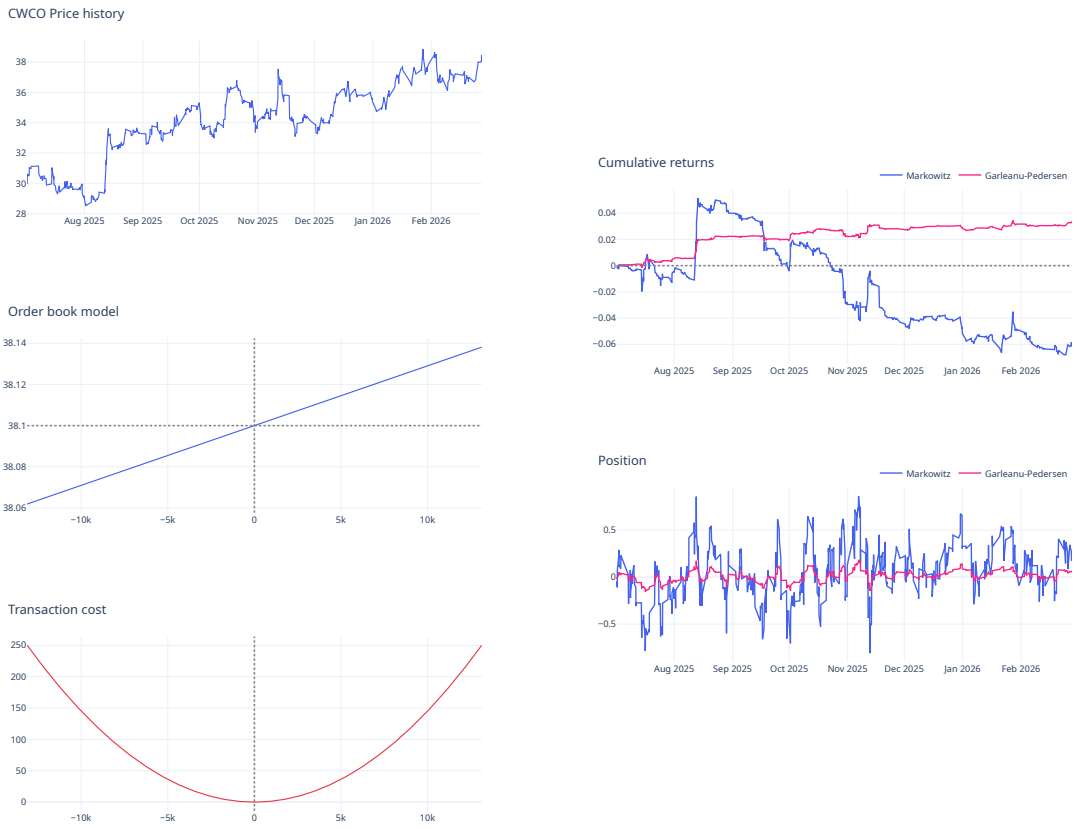

Figure 8: Comparison of the no-transaction-cost (Markowitz) portfolio with the optimal quadratic transaction cost (Gârleanu–Pedersen) portfolio in a purely impact cost setting. The left shows the implied order book model from the cost parameters with AUM of \$10M, and the right shows the total cumulative return over the period along with position sizing over time.

	Ann. Excess Return	Ann. Vol	Sharpe	Max DD
Markowitz	0.0803	0.1039	0.7725	-0.0947
Garleanu-Pedersen	0.0841	0.0947	0.8877	-0.0832

Table 7: Net-of-cost performance comparison of the portfolio strategies: the no-cost (Markowitz) portfolio and the impact-cost-optimal (Gârleanu–Pedersen) portfolio, in terms of arithmetic annualized excess return, volatility, Sharpe ratio, and maximum drawdown.

Figure 9: Comparison of the no-transaction-cost Markowitz portfolio with the optimal No-Trade Band solution in a purely linear cost regime. The left shows the implied trading band and position dynamics, while the right shows cumulative returns over the period along with position sizing over time.

	Ann. Excess Return	Ann. Vol	Sharpe	Max DD
Markowitz	-0.4657	0.1111	-4.1929	-0.1948
No-Trade Band	0.2196	0.0777	2.8266	-0.0233

Table 8: Net-of-cost performance comparison under a linear transaction cost specification, contrasting the frictionless Markowitz portfolio with the optimal No-Trade Band policy. Reported statistics are arithmetic annualized excess return, volatility, Sharpe ratio, and maximum drawdown.

We now see from Tables 7 and 8, and Figures 8 and 9, that both strategies clearly and significantly outperform the baseline, highlighting the importance of optimizing for trading costs when dealing with small-cap stocks or illiquid asset classes in general. A further consideration that underscores this point is that, in practice, finding return-predicting signals is often easier for small-cap stocks due to fewer informed market participants.

4.5 Conclusion

In this paper we have studied and made progress the problem of optimal execution, when faced with both fixed and impact costs. We provide a novel solution to this problem based on recent developments in deep-learning. Our results indicate, that the model significantly outperforms the baseline on simulated data, showing an 8.1 Sharpe ratio improvement. This solution is implemented as a digital twin (DT) trading system that can dynamically fetch and model relevant market data

and optimally execute the given strategy intelligently. We have validated the DT under different scenarios, specifically when liqudity is low. Overall, this provides a novel solution to a complex the complex problem of trading under costs, and a framework for bridging theoretical optimal trading and practical implementation.

5 Authors Contributions

Juba Amari and Halfdan Feldthus jointly conceived the project. Juba Amari developed the theoretical model and wrote Section 2. Halfdan Feldthus developed the software implementation and wrote sections 1, 3, and 4. Both authors contributed equally to writing and editing the manuscript and abstract.

6 Acknowledgments

We thank Narimane Hadj Haouaoui from University of Mons for assistance in preparing the presentation materials.

We thank Lasse Heje Pedersen from AQR Capital management for insightful comments and advice.

Finally, we thank Prof. Giovanna Di Marzo Serugendo and Sedoh Akouete Antonin from University of Geneva for assistance and guidance throughout the whole project and for allowing us to tackle this research problem.

References

[1] Optimal Execution Strategies - Almgren-Chriss Model | QuestDB — questdb.com. `https://questdb.com/glossary/optimal-execution-strategies-almgren-chriss-model/`. [Accessed 14-04-2026].

[2] Streamlit Docs — docs.streamlit.io. `https://docs.streamlit.io/`. [Accessed 14-04-2026].

[3] Martín Abadi et al. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015.

[4] Yacine Aït-Sahalia. Maximum likelihood estimation of discretely sampled diffusions: A closed-form approximation approach. *Econometrica*, 70(1):223–262, 2002.

[5] Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2):5–39, 2000.

[6] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.

[7] Richard Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957.

[8] Joachim de Lataillade, Cyril Deremble, Marc Potters, and Jean-Philippe Bouchaud. Optimal trading with linear costs, 2012.

[9] Jean-Louis Dupré and Donatien Hainaut. Deep learning for continuous-time stochastic control with jumps. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[10] Nicolae Gârleanu and Lasse {Heje Pedersen}. Dynamic trading with predictable returns and transaction costs. *Journal of Finance*, 68(6):2309–2340, December 2013.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[12] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017.

[13] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[14] R. DAVID McLEAN and JEFFREY PONTIFF. Does academic research destroy stock return predictability? *The Journal of Finance*, 71(1):5–31, 2016.

[15] Bernt Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. Springer, Berlin, 6th edition, 2003.

[16] A. Rej, R. Benichou, J. de Lataillade, G. Zérah, and J. Ph. Bouchaud. Optimal trading with linear and (small) non-linear costs, 2016.

[17] Jiongmin Yong and Xun Yu Zhou. *Stochastic Controls: Hamiltonian Systems and HJB Equations*. Springer, New York, 1st edition, 1999.

[18] Álvaro Cartea, Sebastián Jaimungal, and José Penalva. *Algorithmic and High-Frequency Trading*. Cambridge University Press, Cambridge, 2nd edition, 2015.